

МЕТОДИЧЕСКИЕ УКАЗАНИЯ**УП.03. Учебная практика****Этап 6. CI/CD, релиз и инцидент поддержки****Выполнение работ на этапе поддержки ПО компьютерной системы**

Параметр	Значение
Практика	УП.03. Сопровождение и обслуживание программного обеспечения компьютерных систем
Этап 6	CI/CD, релиз и инцидент поддержки
Связь с этапом 2	Проект уже запускается локально через BAT, Makefile, Docker, линтеры и форматтеры
Связь с этапом 3	Есть вариант развертывания: сервис, demo-стенд, PaaS, Docker или готовый build
Связь с этапом 4	Есть результаты тестирования, диагностики качества, баги и дефекты
Связь с этапом 5	Есть перечень рисков безопасности и эксплуатации, часть замечаний можно взять как инцидент
Основной результат	Студент создает issue, воспроизводит проблему, диагностирует ее по логам, исправляет через отдельную ветку, настраивает GitHub Actions, делает commit/PR, обновляет changelog и release notes
Формат сдачи	Ссылка на репозиторий + SUPPORT_REPORT.docx + Issue/PR/Actions + CHANGELOG.md + RELEASE_NOTES.md + скриншоты
Инструменты	GitHub Issues, branches, commits, pull request, GitHub Actions, logs, Makefile/BAT, changelog, release build

Главная мысль этапа:

Студент должен пройти полный цикл сопровождения: проблема -> диагностика -> исправление -> проверка -> релиз. Не просто “дописать функцию”, а оформить работу так, как будто проект уже находится в эксплуатации.

Что должно получиться после этапа

После выполнения этапа в репозитории должна появиться понятная история поддержки: проблема оформлена как задача, исправление выполнено в отдельной ветке, проверка прошла автоматически, изменения описаны в changelog/release notes, а проект подготовлен к релизу.

Что должно быть	Зачем это нужно	Минимальный результат
GitHub Issue	Чтобы проблема или улучшение были оформлены как задача поддержки	Есть issue с описанием, шагами воспроизведения и ожидаемым результатом
Отдельная ветка	Чтобы изменения не делались напрямую в main/master	support/fix-auth-error или feature/add-export
Диагностика по логам	Чтобы студент доказал, что нашел причину, а не угадал	Скриншот логов + краткий вывод
Исправление / улучшение	Чтобы была реальная работа сопровождения	Исправлен баг или добавлено небольшое улучшение
GitHub Actions	Чтобы тесты/сборка запускались автоматически	.github/workflows/ci.yml и успешный workflow run
Pull Request	Чтобы изменение было оформлено как проверяемая работа	PR с описанием, чек-листом и ссылкой на issue
CHANGELOG.md	Чтобы фиксировать историю изменений проекта	Добавлена запись о версии/исправлении
RELEASE_NOTES.md	Чтобы понятно объяснить, что вошло в релиз	Краткое описание изменений, исправлений и проверки
Release build / tag	Чтобы показать готовую версию проекта	Архив релиза, тег v0.1.1 или GitHub Release при возможности

Структура сдачи

Сдаётся репозиторий с реальной историей изменений. Отдельная папка нужна только для отчета и скриншотов-доказательств.

Рекомендуемая структура сдачи

```
УП03_Этап6_Иванов_Иван_Группа/  
|-- repo_link.txt  
|-- docs/  
|   |-- SUPPORT_REPORT.docx  
|   |-- INCIDENT_REPORT.md  
|   |-- RELEASE_NOTES.md  
|   |-- RELEASE_CHECKLIST.md  
|   `-- CHANGELOG.md  
|-- screenshots/  
|   |-- 01_github_issue_created.png  
|   |-- 02_branch_created.png  
|   |-- 03_bug_reproduced.png  
|   |-- 04_logs_diagnostics.png  
|   |-- 05_fix_commit.png  
|   |-- 06_tests_passed_locally.png  
|   |-- 07_github_actions_success.png  
|   |-- 08_pull_request.png  
|   |-- 09_changelog_release_notes.png  
|   `-- 10_release_tag_or_archive.png  
|-- project_files_in_repository/  
|   |-- .github/workflows/ci.yml  
|   |-- CHANGELOG.md  
|   |-- RELEASE_NOTES.md  
|   |-- scripts/  
|   |   |-- test.bat  
|   |   |-- build.bat  
|   |   |-- release-check.bat  
|   |   `-- create-release.bat  
|   `-- README.md / DEPLOYMENT.md
```

Порядок выполнения

Шаг	Что сделать	Что приложить
1	Выбрать одну реальную проблему или улучшение проекта. Можно взять дефект из этапа 4 или риск из этапа 5.	Краткое описание выбранной проблемы в SUPPORT_REPORT.docx
2	Создать GitHub Issue. Указать: описание, шаги воспроизведения, ожидаемый результат, фактический результат, окружение.	01_github_issue_created.png
3	Создать отдельную ветку для исправления: support/fix-... или feature/... .	02_branch_created.png
4	Воспроизвести проблему. Показать ошибку в интерфейсе, терминале, Postman, логах или тесте.	03_bug_reproduced.png
5	Провести диагностику: посмотреть логи, код, запросы, настройки, зависимости. Записать причину.	04_logs_diagnostics.png + INCIDENT_REPORT.md
6	Внести исправление, рефакторинг или небольшое улучшение. Изменение должно быть видно в commit.	05_fix_commit.png
7	Запустить локальные проверки: тесты, линтер, сборка, release-check.	06_tests_passed_locally.png
8	Настроить GitHub Actions для автоматической проверки проекта.	07_github_actions_success.png
9	Создать Pull Request. В описании указать issue, что изменено и как проверено.	08_pull_request.png
10	Обновить CHANGELOG.md, RELEASE_NOTES.md и подготовить release build/tag/archive.	09_changelog_release_notes.png, 10_release_tag_or_archive.png

Что можно выбрать как инцидент поддержки

Тип задачи	Примеры	Что считается результатом
Баг	Не работает авторизация; не сохраняется форма; падает API; не открывается страница; ошибка подключения к БД	Ошибка воспроизведена, причина найдена, исправление выполнено, проверка прошла
Технический долг	Дублирование кода, плохая структура, устаревший конфиг, некорректная обработка ошибок	Код стал чище, поведение проекта не сломалось, тесты/сборка проходят
Небольшая функция	Экспорт данных, фильтр, новый статус, новая страница, улучшенный вывод ошибок	Функция работает и описана в release notes
Улучшение эксплуатации	Добавить healthcheck, улучшить логи, добавить команду backup, исправить Docker/Compose	Проект удобнее сопровождать и проверять
Риск из этапа 5	Секрет в репозитории, нет .gitignore, нет backup, открытый debug, слабая обработка доступа	Риск устранен или снижена вероятность/последствия

Важно:

Не принимается “исправление”, которое нельзя увидеть. У каждого изменения должен быть issue, ветка, commit, проверка и понятное описание результата.

Шаблон GitHub Issue

```
# Проблема: краткое название

## Описание
Что произошло и почему это мешает работе проекта.

## Шаги воспроизведения
1. Открыть ...
2. Выполнить ...
3. Получить ошибку ...

## Ожидаемый результат
Что должно происходить.

## Фактический результат
Что происходит сейчас.

## Окружение
- ОС:
- Браузер / версия Python / Node / Java:
- Режим запуска: local / Docker / demo / production

## Доказательства
Скриншот, лог, ссылка на тест или фрагмент ошибки.
```

Шаблон INCIDENT_REPORT.md

```
# INCIDENT_REPORT.md

## 1. Инцидент
Название проблемы или обращения пользователя.

## 2. Где обнаружено
Локально / Docker / демо-стенд / deployed версия.

## 3. Как воспроизвести
Пошаговое описание.

## 4. Диагностика
Какие логи, файлы, запросы, тесты были проверены.

## 5. Причина
Почему возникла проблема.

## 6. Исправление
Какие файлы изменены и почему.

## 7. Проверка
Какие команды выполнены: make check, scripts\test.bat, docker compose logs, GitHub Actions.

## 8. Итог
Проблема исправлена / частично исправлена / требуется доработка.
```

БАТ-скрипты этапа

Скрипты нужны, чтобы проверка этапа не зависела от памяти студента. Преподаватель должен запустить одну команду и увидеть результат.

scripts/test.bat

```
# scripts/test.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."
echo Запуск тестов и проверок
make check
```

scripts/build.bat

```
# scripts/build.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."
echo Сборка проекта
make build
```

scripts/release-check.bat

```
# scripts/release-check.bat
@echo off
chcp 65001 > nul
cd /d "%~dp0\.."
echo Финальная проверка перед релизом
make check
make build
```

Makefile: команды поддержки и релиза

```
.PHONY: check test build release-check logs clean

check:
    @echo "Run linter, tests or build checks"
    # пример: pytest
    # пример: npm run lint && npm run build
    # пример: mvn test

test:
    @echo "Run tests"
    # команда тестирования проекта

build:
    @echo "Build release version"
    # команда сборки проекта

release-check: check build
    @echo "Project is ready for release"

logs:
    docker compose logs --tail=100

clean:
    @echo "Clean temporary files if it is safe"
```

GitHub Actions: минимальный CI

CI должен запускаться при push и pull request. Минимум: установить зависимости и выполнить check/test/build. Студент выбирает вариант под свой стек.

```
# .github/workflows/ci.yml
name: CI

on:
  push:
  pull_request:

jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Set up Python
        if: hashFiles('requirements.txt') != ''
```

```

uses: actions/setup-python@v5
with:
  python-version: '3.12'

- name: Install Python dependencies
  if: hashFiles('requirements.txt') != ''
  run: pip install -r requirements.txt

- name: Python check
  if: hashFiles('requirements.txt') != ''
  run: |
    python -m pytest || true
    python -m compileall .

- name: Set up Node
  if: hashFiles('package.json') != ''
  uses: actions/setup-node@v4
  with:
    node-version: '20'

- name: Node install and check
  if: hashFiles('package.json') != ''
  run: |
    npm install
    npm run check --if-present
    npm run build --if-present

```

Уточнение:

Если проект на Java, C#, Android или другом стеке, GitHub Actions можно адаптировать. Главное: workflow должен реально запускаться и завершаться успешно либо показывать ошибку, которую студент исправил.

Pull Request: что должно быть в описании

```

# Что изменено
- Исправлена проблема ...
- Обновлена обработка ошибки ...
- Добавлена проверка ...

# Связанный issue
Closes #номер_issue

# Как проверено
- scripts\test.bat
- make check
- GitHub Actions CI
- ручная проверка сценария: ...

# Скриншоты
Добавлены в папку screenshots этапа 6.

```

CHANGELOG.md и RELEASE_NOTES.md

Файл	Что написать	Пример
CHANGELOG.md	Короткая техническая история изменений по версиям	v0.1.1 — исправлена ошибка авторизации, добавлен healthcheck, обновлен Dockerfile
RELEASE_NOTES.md	Человеческое описание релиза: что изменилось, как проверить, есть ли ограничения	Релиз исправляет ошибку входа и улучшает стабильность запуска в Docker
RELEASE_CHECKLIST.md	Список финальных проверок перед сдачей	Тесты пройдены, CI зеленый, release build создан, демо работает

Пример CHANGELOG.md

```

# CHANGELOG.md

## [0.1.1] - 2026-05-31
### Fixed
- Исправлена ошибка при ...

### Changed
- Улучшена обработка ...
- Обновлено команды запуска/проверки.

### Verified
- Локальные проверки: make check
- CI: GitHub Actions passed

```

Пример RELEASE_NOTES.md

```
# RELEASE_NOTES.md

## Релиз 0.1.1

### Что изменилось
Краткое описание изменений простым языком.

### Что исправлено
Описание исправленного инцидента.

### Как проверить
1. Запустить проект.
2. Выполнить основной сценарий.
3. Убедиться, что ошибка больше не воспроизводится.

### Ограничения
Что еще требует доработки.
```

Обязательные скриншоты

Файл	Что должно быть видно
01_github_issue_created.png	Созданный GitHub Issue с описанием проблемы или улучшения
02_branch_created.png	Отдельная ветка support/... или feature/...
03_bug_reproduced.png	Ошибка, баг или проблема воспроизведены
04_logs_diagnostics.png	Логи, ошибка в консоли, Postman, DevTools или терминале
05_fix_commit.png	Commit с исправлением или улучшением
06_tests_passed_locally.png	Локальная проверка: make check / scripts/test.bat / сборка
07_github_actions_success.png	Успешное выполнение GitHub Actions
08_pull_request.png	Pull Request с описанием и ссылкой на issue
09_changelog_release_notes.png	Обновленные CHANGELOG.md и RELEASE_NOTES.md
10_release_tag_or_archive.png	Тег релиза, GitHub Release или архив релизной сборки

Короткий отчет SUPPORT_REPORT.docx

Раздел отчета	Что написать
1. Ссылка на репозиторий	Ссылка на GitHub/GitLab и ветка с работой этапа
2. Какая проблема выбрана	1-2 абзаца: что сломано или что нужно улучшить
3. Как воспроизводится проблема	Короткие шаги воспроизведения
4. Что показали логи/диагностика	Какая причина найдена
5. Что изменено в проекте	Список измененных файлов и смысл изменений
6. Как проверено	Команды, тесты, GitHub Actions, ручная проверка
7. Что вошло в релиз	Номер версии, changelog, release notes, архив/тег
8. Скриншоты	Список скриншотов с кратким пояснением

Критерии приемки этапа

Критерий	Принимается	Не принимается
Issue	Есть полноценное описание проблемы/улучшения со шагами воспроизведения	Issue отсутствует или содержит одну фразу
Ветка	Работа выполнена в отдельной ветке	Все изменения сделаны напрямую в main/master
Диагностика	Есть логи, ошибка, причина и вывод	Студент пишет “исправил”, но не показывает причину
Исправление	Есть конкретный commit с изменениями	Изменения не связаны с заявленной проблемой
Проверка	Локальные проверки и GitHub Actions выполнены	Проверки не запускались
PR	Есть Pull Request или оформленный merge с описанием	Нет истории проверки изменений
Релиз	Есть changelog, release notes, build/tag/archive	Нет итоговой версии изменений
Скриншоты	Скриншоты доказывают реальные действия	Скриншоты формальные и не подтверждают работу

Минимум для зачета:

Один issue, одна ветка, одно воспроизведение проблемы, один осмысленный commit, одна успешная проверка, один Pull Request или оформленный merge, обновленный changelog/release notes и скриншоты доказательств.